Resampling Methods

Cross-Validation for Tuning, Bootstrap for Uncertainty

Tu Li

Fall 2026

Econ 453 University of Arizona

Roadmap

1. The Tuning-Parameter Problem

1.1 Three Loose Ends from L10–L12

Each of the last three lectures introduced a model that depends on a tuning parameter we promised to pick later:

- L10 (Ridge / LASSO). The penalty weight $\lambda \geq 0$ controls how much we shrink coefficients.
- L11 (Polynomial regression). The degree d controls the flexibility of $\hat{f}$.
- L12 (LASSO logistic). Same λ as in L10, now attached to the negative log-likelihood.

In each case we wrote: "treat λ (or d) as given for now; we will pick it from the data later."

Today we deliver. One procedure — K-fold cross-validation — picks all of them.

1.2 The Unifying Goal

All three tuning parameters control a single tradeoff:

- Small λ / large d: flexible model. Low bias, high variance.
- Large λ / small d: stiff model. High bias, low variance.

We want the value of the tuning parameter that minimises the test error — the expected squared prediction error on a fresh, independent observation.

We do not have access to fresh data. We have only the training data we used to fit the model. The trick is to use the training data to *estimate* the test error.

1.3 Roadmap of the Lecture

Two halves with different purposes.

Part I (§2–§6): Cross-Validation
Estimate the test error of a model. Use the estimate to pick a tuning parameter. Three worked examples on variable selection, LASSO/ridge, and polynomial regression.

Part II (§7–§12): Bootstrap
Estimate the standard error of *any* statistic — coefficient, R^2, median, prediction — when no closed-form SE formula exists. Formula-free, applies to any method in the course.

Both halves rely on the same fundamental idea: re-use the data we have to learn what we would have learned from fresh data.

2. Training Error vs Test Error

2.1 Two Different Errors

Definitions
Let $\hat{f}$ be a model fitted on training data $\{(x_i, y_i)\}_{i=1}^{n}$.

Training error: the average squared residual on the very same data used to fit $\hat{f}$,

$$\text{Train}(\hat{f}) \;=\; \frac{1}{n}\sum_{i=1}^{n}\big(y_i - \hat{f}(x_i)\big)^2.$$

Test error: the expected squared error on a fresh, independent observation (x_0, y_0) drawn from the same distribution as the training data,

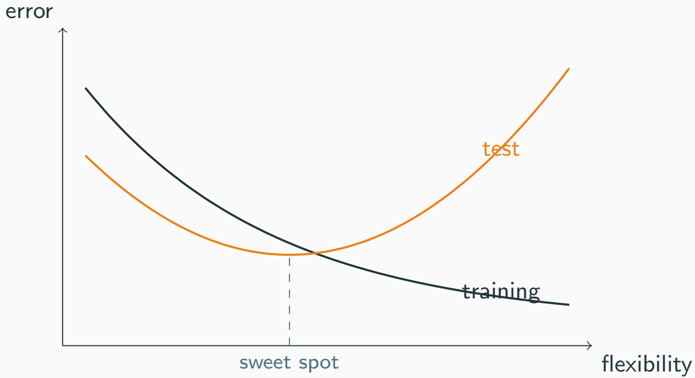
$$\text{Test}(\hat{f}) \;=\; \mathbb{E}\big[(y_0 - \hat{f}(x_0))^2\big].$$

2.1 Two Different Errors 2

Notation reminder. $\hat{f}$ is held fixed at the value computed from training data; (x_0, y_0) is the random fresh point. Expectation in $\text{Test}(\hat{f})$ is over (x_0, y_0), not over the training data.

2.2 The Picture That Matters

Plot both errors as functions of model flexibility (e.g. polynomial degree d, or number of predictors p).



2.2 The Picture That Matters 2

Training error keeps falling as the model gets more flexible — just like R^2 always rises with p (L9 §6). Test error U-shapes: bias falls, variance rises (L10 §3).

2.3 Why Training Error Is Optimistic

The same data that fit $\hat{f}$ are also the data scored against $\hat{f}$.

Adding flexibility lets $\hat{f}$ "chase" the training residuals more aggressively. Eventually $\hat{f}$ fits the random noise in the training points, not just the signal.

Performance on fresh data — which has *different* noise — cannot benefit from chasing the old noise. So

$$\mathrm{Train}(\hat{f}) \;\le\; \mathrm{Test}(\hat{f}),$$

often dramatically so for flexible models.

Bottom line. Picking a model to minimise training error always picks the most flexible model on the menu. That is overfitting.

2.4 What We Need

We cannot observe the test error directly (we don't have a fresh data set).

But we *do* have data. Can we use part of the data we have to simulate "fresh" data?

Two ideas follow:

- Validation-set approach (§3): split the data once into a training half and a held-out half.
- K-fold cross-validation (§4): split the data K ways and average. This is the workhorse.

3. The Validation-Set Approach

3.1 The Simplest Fix

Randomly split the n observations into two parts:

- Training set (say 70%): used to fit $\hat{f}$.
- Validation set (say 30%): used to score $\hat{f}$.

Compute the validation MSE:

$$\mathrm{Val}(\hat{f}) \;=\; \frac{1}{n_{\mathrm{val}}} \sum_{i \in \mathcal{V}} \big(y_i - \hat{f}(x_i)\big)^2,$$

where $\mathcal{V}$ is the index set of the validation half.

This is an estimate of $\mathrm{Test}(\hat{f})$.

3.2 Why It Is (In Principle) Sound

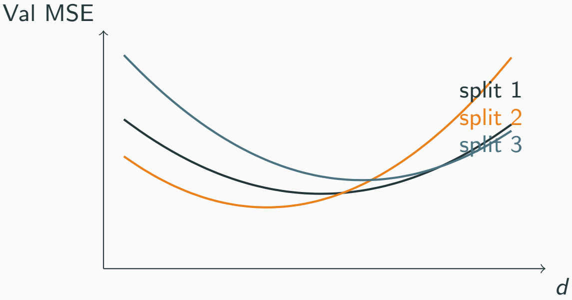
The validation observations were not used to fit $\hat{f}$. From the model's point of view they are fresh.

The cross-term that made training error optimistic disappears: scoring a model on data it has never seen does not benefit from chasing those points' noise.

In symbols: $\mathbb{E}[\mathrm{Val}(\hat{f})] \approx \mathrm{Test}(\hat{f})$, with the approximation getting better as the validation set grows.

3.3 Drawback 1 — High Variance

Two different random splits give two different validation MSEs.



3.3 Drawback 1 — High Variance 2

All three curves are valid estimates. Which d minimises them differs by split — the choice depends on luck of the draw.

Lesson: a single validation split is a high-variance estimator of test error.

3.4 Drawback 2 — Wasted Data

We only used $0.7n$ observations to fit $\hat{f}$. The remaining $0.3n$ never contribute to estimation.

With $n = 200$ that costs us 60 observations. With small samples, that is a serious loss of information about f.

Moreover, the model $\hat{f}$ we end up choosing is fit on $0.7n$ observations but eventually deployed using all n. The test error we estimated is for the smaller-sample fit, not the final-sample fit.

3.5 What We Need

We want a procedure that:

1. Uses every observation for *both* fitting and validation (no wasted data).
2. Averages over many splits to reduce the variance of the MSE estimate.
3. Still keeps fitting and validation cleanly separated (no leakage).

That procedure is K-fold cross-validation.

4. K-Fold Cross-Validation

4.1 The K-Fold Algorithm

Definition: K-fold cross-validation
Partition the data into K equal-sized disjoint subsets (folds) $\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_K$. For each $k = 1, 2, \ldots, K$:

1. Fit $\hat{f}^{(-k)}$ on the $K-1$ folds excluding $\mathcal{D}_k$.
2. Score on the held-out fold:

$$\text{MSE}_k \;=\; \frac{1}{|\mathcal{D}_k|} \sum_{i \in \mathcal{D}_k} \big(y_i - \hat{f}^{(-k)}(x_i)\big)^2.$$

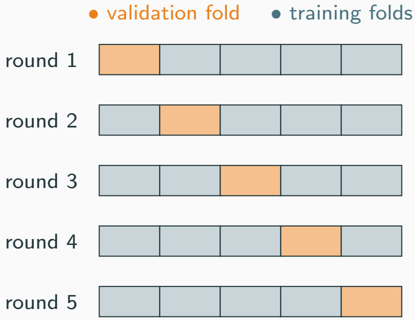
The K-fold CV estimate of the test error is the average

$$\text{CV}_K \;=\; \frac{1}{K} \sum_{k=1}^{K} \text{MSE}_k.$$

4.1 The K-Fold Algorithm 2

Notation reminder. $\hat{f}^{(-k)}$ is the model fitted on the data *minus* fold k. We end up fitting K different models, one per fold.

4.2 The 5-Fold Partition Diagram



Each row is one of the $K = 5$ CV rounds. The shaded cell is the held-out fold for that round; the other four are training data.

Across all K rounds, every observation lands in the validation fold exactly once and the training folds $K - 1$ times.

4.3 Why K-Fold Is Efficient

Compared to a single 70/30 validation split:

- No wasted data. Every observation contributes to both fitting and validation over the course of the procedure.
- Lower variance. Averaging K MSEs cancels out the luck-of-the-split noise that contaminates any single 70/30 estimate.
- Larger training sets. Each $\hat{f}^{(-k)}$ is fit on $(K-1)/K \approx 80\%$ or 90% of the data (for $K = 5$ or 10), much closer to the final-sample fit than a 70% split.

The price is K refits instead of one. Computers handle that.

4.4 LOOCV — The $K = n$ Special Case

Push K all the way to n: each fold contains a single observation. This is Leave-One-Out Cross-Validation (LOOCV).

$$\mathrm{CV}_n = \frac{1}{n}\sum_{i=1}^{n}\big(y_i - \hat{f}^{(-i)}(x_i)\big)^2,$$

where $\hat{f}^{(-i)}$ is the model fit on the data minus observation i.

Pros and cons:

- Low bias. Training-set size $n-1 \approx n$, so $\hat{f}^{(-i)}$ behaves almost identically to the full-sample fit.
- High variance. The n leave-one-out fits are almost identical to each other (each differs by one observation), so the n residuals are highly correlated. Their average doesn't gain as much noise-cancellation as K-fold with smaller K.

4.4 LOOCV — The $K = n$ Special Case 2

- Expensive. n refits. Painful for large n or slow-to-fit models.

Practical recommendation. Use $K = 5$ or $K = 10$. Sweet spot of bias, variance, and compute. "Computer does it" — standard software ships with these defaults.

4.5 The Selection Rule

Suppose we have a family of candidate models indexed by a tuning parameter θ (could be λ in LASSO, d in polynomial regression, or a discrete subset choice).

Compute the K-fold CV estimate $\mathrm{CV}_K(\hat{f}_\theta)$ for each candidate. Pick the minimiser:

$$\hat{\theta}_{\mathrm{CV}} \;=\; \arg\min_{\theta} \mathrm{CV}_K(\hat{f}_\theta).$$

Notation reminder. $\hat{f}_\theta$ is the model *family* parametrised by θ. Inside the CV procedure, for each θ on our grid we fit K versions of $\hat{f}_\theta$ (one per fold), score them, and average.

Three uses of this rule follow.

5. Three CV Examples

5.1 Variable Selection — Setup

Recall L10 §2: with p candidate predictors there are 2^p possible subsets (including the empty model).

When p is small — say $p \leq 10$ or so — we can afford to enumerate every subset. For each subset S:

1. Fit OLS using only the predictors in S.
2. Run K-fold CV; record $\mathrm{CV}_K(S)$.

Pick the subset with smallest CV-MSE:

$$\hat{S}_{\mathrm{CV}} \;=\; \operatorname*{arg\,min}_{S \subseteq \{1,\ldots,p\}} \; \mathrm{CV}_K(S).$$

5.2 Variable Selection — Worked Example

Re-using the L10 §6 used-cars setup but restricted to $p = 5$ predictors: `mileage`, `age`, `horsepower`, `accident`, `luxury`. There are $2^5 = 32$ candidate models. Below are the top six by CV-MSE (10-fold, $n = 200$).

#	Predictors in model	CV-MSE ($\times 10^6$ USD^2)
1	mileage, age, horsepower, luxury	3.9
2	mileage, age, horsepower	4.1
3	mileage, age, horsepower, accident, luxury	4.2
4	mileage, age, luxury	4.5
5	mileage, horsepower	4.9
6	age, horsepower, luxury	5.1

5.2 Variable Selection — Worked Example 2

Winner: the four-predictor model (mileage, age, horsepower, luxury), matching the best-4 model in L10 §6. Adding `accident` slightly increases CV-MSE: it did not improve held-out prediction enough to justify the extra parameter.

Note. Training MSE would have monotonically improved as predictors were added. CV-MSE penalises the extra parameter via variance on held-out folds. This is the entire point.

5.3 Variable Selection — Limits

For $p = 5$, $2^5 = 32$ subsets. Fine.

For $p = 20$, $2^{20} \approx 10^6$ subsets. Painful but possible.

For $p = 50$ or beyond, 2^p is astronomical. Best subset is infeasible.

L10 §5 already gave us the fix: LASSO performs variable selection in a single fit, no exhaustive enumeration required. And LASSO comes with its own tuning parameter λ, which CV can pick. That is the next example.

5.4 Picking λ — Grid Setup

Pick a grid of candidate λ values, typically equally spaced on the log scale:

$$\lambda \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 10^{2}, 10^{3}\}.$$

For each grid value:

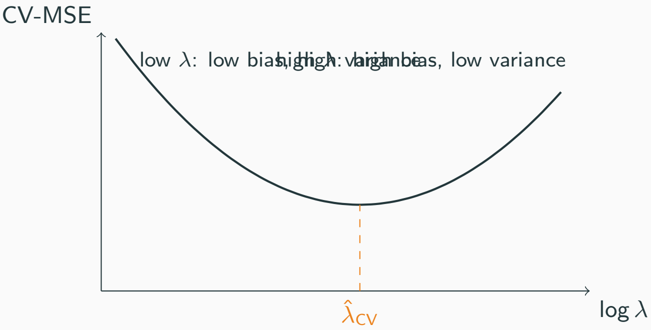
1. For each fold k, fit LASSO at this λ on the $K-1$ training folds.
2. Score on the held-out fold; record $\mathrm{MSE}_k(\lambda)$.
3. Average over folds: $\mathrm{CV}_K(\lambda)$.

Pick $\hat{\lambda}_{\mathrm{CV}} = \arg\min_{\lambda} \mathrm{CV}_K(\lambda)$.

As in L10, only the slope coefficients are penalised; the intercept is left unpenalised in every fold.

"Computer does it." Software ships with this loop built in.

5.5 Picking λ — The U-Curve



This is the L10 §3.5 bias–variance U-curve, now indexed by λ and estimated from data rather than drawn from theory.

$\hat{\lambda}_{\text{CV}}$ is the value at the bottom of the U.

5.6 Picking λ — Worked Example

Same fabricated $n = 200$ used-cars data as L10 §6, with all $p = 8$ predictors. 10-fold CV at five grid points:

λ	CV-MSE ($\times 10^6$ USD^2)	# nonzero slopes
0.1	5.1	8
1.0	4.6	8
10	3.7	7
100	3.2	5
1000	4.9	1

Minimum at $\hat{\lambda}_{\text{CV}} = 100$. At that value LASSO has zeroed out three predictors (`mpg_city`, `mpg_hwy`, `n_owners` — exactly the pattern reported in L10 §6.3).

5.6 Picking λ — Worked Example 2

Delivering on the L10 §4.8 promise
The hyperparameter λ that L10 left as "picked from data later" is picked right here. Same recipe works for ridge — just swap the penalty inside the fit step. Same recipe works for LASSO logistic (L12 §8) — swap RSS for negative log-likelihood inside the fit step. One procedure, three uses.

5.7 Picking λ — The "One Standard Error" Rule

Many software packages report two choices instead of one:

- $\hat{\lambda}_{\text{min}}$: the grid value minimising CV_K.
- $\hat{\lambda}_{\text{1se}}$: the largest λ whose CV_K is within one standard error of the minimum.

The 1-SE rule trades a small predicted MSE penalty for a more parsimonious model (more zero slopes in LASSO). When the CV-MSE curve is flat near its minimum, the parsimony is essentially free.

We won't derive it. Just know that "computer reports two numbers; either is defensible".

5.8 Picking d — Grid Setup

From L11 §5: polynomial regression of degree d fits

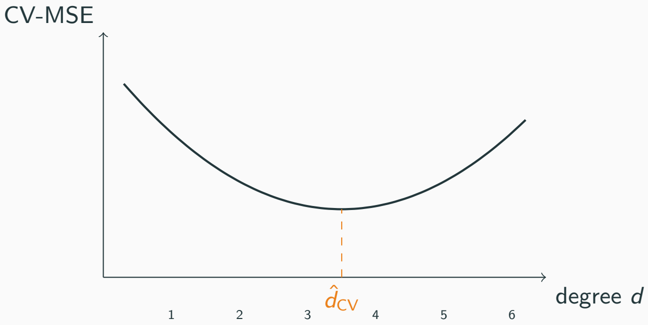
$$\hat{f}(x) \;=\; \hat{\beta}_0 + \hat{\beta}_1 x + \hat{\beta}_2 x^2 + \cdots + \hat{\beta}_d x^d$$

by OLS on the design matrix $\mathbf{X}_{\text{poly}}$.

d is a tuning parameter: small d underfits, large d overfits (L11 §5.4).

Pick a grid, e.g. $d \in \{1, 2, 3, \ldots, 10\}$, run 10-fold CV at each, pick the minimiser.

5.9 Picking d — The U-Curve



Same U-shape as §5.5. Different tuning parameter; identical procedure.

This is the spine of the lecture: one procedure picks every tuning parameter we have left dangling.

5.10 Picking d — Worked Example

Re-using the L11 §5.5 setup: $n = 100$, x_i uniform on $[-3, 3]$, and $y_i = \sin(x_i) + 0.3\epsilon_i$ with $\epsilon_i \sim N(0, 1)$. Run 10-fold CV on polynomial fits at $d \in \{1, \ldots, 10\}$:

d	CV-MSE
1	0.51
2	0.48
3	0.15
4	0.10
5	0.11
6	0.11
7	0.12
8	0.13
9	0.15
10	0.18

5.10 Picking d — Worked Example 2

Winner: $\hat{d}_{\text{CV}} = 4$. Exactly the choice L11 §5.5 declared by eye; now picked procedurally.

Delivering on the L11 §3.3 promise
The hyperparameter d that L11 introduced and left as "picked from data later" is picked right here. Same K-fold CV recipe; just a different grid.

6. Cross-Validation Summary

6.1 One Procedure, Many Uses

We have used K-fold CV three times in three places:

- §5.1–§5.3: pick a discrete subset of predictors.
- §5.4–§5.7: pick a continuous λ for LASSO or ridge.
- §5.8–§5.10: pick a discrete degree d for polynomial regression.

Same recipe each time:

1. Define a grid of candidate values for the tuning parameter.
2. For each grid value, run K-fold CV; record CV_K.
3. Pick the minimiser.

Any tuning parameter, in any method, in any of our remaining lectures, is picked this way. This is the universal procedure.

6.2 The Practitioner's Recipe

Recipe for picking any tuning parameter

1. Choose K (usually 5 or 10).
2. Partition the data into K equal folds.
3. For each candidate tuning value θ on your grid:
    - For each $k = 1, \dots, K$: fit on the $K - 1$ folds excluding $\mathcal{D}_k$, score on $\mathcal{D}_k$.
    - Average the K MSEs.
4. Pick $\hat{\theta}$ minimising the average.
5. Refit using $\hat{\theta}$ on the full data.

"Computer does it" — step 3 is one function call in any modern ML library.

6.3 Forward Pointer to Part II

Cross-validation estimates the expected test error of a model. That is one kind of question:

- "Which λ gives the best predictions on average?"

A different question is about the variability of a statistic, not its expected value:

- "How precisely is $\hat{\beta}_j$ pinned down by the data?"
- "What is a 95% confidence interval for the predicted price at a new $\mathbf{x}_0$?"

Standard errors and confidence intervals. We have closed-form formulas for some of these (L7 SLR, L9 OLS). For most we don't.

Part II of the lecture handles "most."

7. Motivation — Beyond Closed-Form Standard Errors

7.1 Recall — The Sample Mean

From L3: if $Y_1, \ldots, Y_n$ are iid with variance σ^2, then the sample mean $\bar{Y}$ has standard error

$$\mathrm{SE}(\bar{Y}) \;=\; \frac{\sigma}{\sqrt{n}}.$$

Closed form. Universal — works for any underlying distribution F with finite variance. Estimated by $\hat{\mathrm{SE}}(\bar{Y}) = s/\sqrt{n}$ (replace σ with the sample standard deviation).

Most of L3, L4, L7, and L9 was built on this kind of closed-form SE. Today we go beyond it.

7.2 Notation Reminder — The Sample Median

We have not formally defined the median before, so let's do so now.

Definition: sample median
Sort the observations from smallest to largest:

$$y_{(1)} \leq y_{(2)} \leq \cdots \leq y_{(n)}.$$

The sample median $\tilde{m}$ is the middle value:

- If n is odd: $\tilde{m} = y_{((n+1)/2)}$.
- If n is even: $\tilde{m} = \frac{1}{2}\left(y_{(n/2)} + y_{(n/2+1)}\right)$.

Worked example with $n = 7$. Data: $\{8, 3, 11, 7, 2, 9, 5\}$. Sorted: $\{2, 3, 5, 7, 8, 9, 11\}$. Middle value is the 4th: $\tilde{m} = 7$.

7.2 Notation Reminder — The Sample Median 2

Worked example with $n = 6$. Data: $\{8, 3, 11, 7, 2, 9\}$. Sorted: $\{2, 3, 7, 8, 9, 11\}$. Middle two are the 3rd and 4th: 7 and 8. Average: $\tilde{m} = \frac{1}{2}(7 + 8) = 7.5$.

The sample median is robust: a single huge outlier does not budge it (try replacing the largest value with 10^9; $\tilde{m}$ is unchanged). This is one reason it's a popular statistic.

7.3 What Is $\mathrm{SE}(\tilde{m})$?

Theory gives an asymptotic formula:

$$\mathrm{SE}(\tilde{m}) \;\approx\; \frac{1}{2\, f(m)\, \sqrt{n}},$$

where m is the *population* median and f is the population *density* at m.

Two problems:

- We don't know f.
- We don't know $f(m)$ even if we knew f somewhere else.

No usable closed form for $\mathrm{SE}(\tilde{m})$.

Plug-in estimators of $f(m)$ exist but are awkward; they require more advanced (non-parametric density estimation) machinery than this course covers.

We need a different tool.

7.4 The General Problem

The sample median is one example. Many statistics we care about share its fate — no clean closed-form SE:

- Quantiles in general (25th percentile, 90th percentile).
- Ratios of estimates: $\hat{\theta}_1/\hat{\theta}_2$.
- Predicted values from non-linear methods (LASSO, polynomial regression at large d).
- R^2 in a regression.
- $\hat{\beta}_j$ when observations are clustered or time-dependent, or when errors are heteroscedastic (L7's basic SE assumes independent observations and constant conditional variance).
- Any composite statistic of an estimator.

We want a generic procedure that produces an SE for *any* computable statistic, with no per-statistic derivation.

That is the bootstrap.

8. The Bootstrap Algorithm

8.1 The Idea — Resample With Replacement

We don't have a fresh dataset. We have the observed data.

Bootstrap principle. Treat the observed data as a stand-in for the population. Draw new datasets from the observed data — with replacement — and recompute the statistic on each.

The resulting collection of resampled statistics is an empirical approximation to the sampling distribution of $\hat{\theta}$.

All we need is the ability to:

1. Draw random integers from $\{1, \ldots, n\}$ with replacement.
2. Compute $\hat{\theta}$ from a data set.

Definition: bootstrap sample
Let $\{y_1, \ldots, y_n\}$ be the observed data. A bootstrap sample of size n is

$$y_1^*, y_2^*, \ldots, y_n^*,$$

where each y_i^* is drawn independently and uniformly at random, with replacement, from $\{y_1, \ldots, y_n\}$.

Consequence: some originals appear more than once in a single bootstrap sample; some don't appear at all. On average each original appears ≈ 1 time, but the variability across bootstrap samples is what powers the procedure.

Small example. Original data $\{2, 3, 7, 8, 9\}$ ($n = 5$). Three example bootstrap samples:

8.2 Definition — Bootstrap Sample 2

- $\{2, 3, 3, 8, 9\}$ — 3 appears twice, 7 missing.
- $\{8, 9, 9, 9, 2\}$ — 9 appears three times.
- $\{7, 7, 7, 7, 7\}$ — legal but very unlikely.

For regression problems with predictors, a bootstrap sample of the rows (x_i, y_i) keeps each (x, y) pair intact and resamples whole rows.
"Computer does it."

8.3 The Bootstrap Algorithm

Algorithm: bootstrap
Fix B (the number of bootstrap replicates — typically $B = 1000$ or 10000). For $b = 1, 2, \ldots, B$:

1. Draw a bootstrap sample of size n from the original data.
2. Compute the statistic of interest on this sample: $\hat{\theta}^{*b}$.

Output: the bootstrap distribution $\{\hat{\theta}^{*1}, \hat{\theta}^{*2}, \ldots, \hat{\theta}^{*B}\}$.

This collection is the empirical analogue of a sampling distribution. "As if" we had replicated the original experiment B times and got a different $\hat{\theta}$ from each replication.

Everything we want to know about $\hat{\theta}$ — its standard error, confidence intervals, hypothesis-test p-values — we read off this collection.

8.4 Bootstrap Standard Error

Definition: bootstrap SE

$$\widehat{\mathsf{SE}}_{\text{boot}}(\hat{\theta}) \;=\; \sqrt{\frac{1}{B-1}\sum_{b=1}^{B}\big(\hat{\theta}^{*b} - \bar{\hat{\theta}}^{*}\big)^2}, \qquad \bar{\hat{\theta}}^{*} \;=\; \frac{1}{B}\sum_{b=1}^{B}\hat{\theta}^{*b}.$$

This is the sample standard deviation of the bootstrap distribution — the very same sample-variance formula from L3 §7.1, applied to the B resampled statistics.

No properties of $\hat{\theta}$ were needed: no derivatives, no closed forms, no asymptotics. Just resample-and-recompute.

8.5 How Many Resamples B?

Two sources of error in the bootstrap procedure:

- Statistical error. The bootstrap distribution approximates the true sampling distribution well only if n is reasonably large. This is fixed by data, not by us.
- Monte Carlo error. The B bootstrap replicates themselves give a finite-B estimate. This shrinks as B grows.

Standard practice:

- $B = 1000$ is plenty for an SE or a confidence interval.
- $B = 10000$ for hypothesis-test p-values (need accuracy in the tails).

Going beyond $B = 10^4$ rarely changes any reported number to more than three significant figures. “Computer does it” — the loop is embarrassingly parallel.

9. Bootstrap Example 1 — Confidence Intervals

9.1 The Bootstrap Percentile CI

Definition: bootstrap percentile CI
A $100(1-\alpha)\%$ confidence interval for θ is

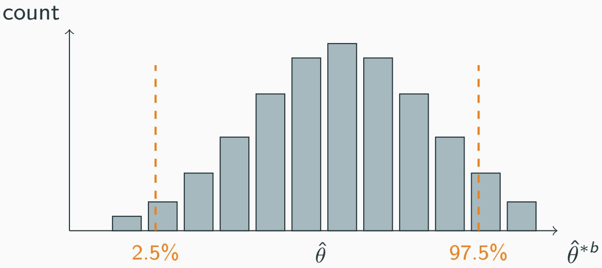
$$\left[\hat{\theta}^*_{(\alpha/2)},\ \ \hat{\theta}^*_{(1-\alpha/2)}\right],$$

where $\hat{\theta}^*_{(q)}$ is the q-quantile of the bootstrap distribution $\{\hat{\theta}^{*b}\}_{b=1}^{B}$.

For 95% CI: take the 2.5% and 97.5% quantiles of the B resampled statistics.

No SE formula, no t-multiplier, no normality assumption. The CI is read directly off the histogram of bootstrap replicates.

9.2 Picture



Each bar is the count of bootstrap replicates falling in a small range. The two dashed lines are the 2.5% and 97.5% quantiles — the endpoints of the 95% CI.

9.3 Worked Example — LASSO Coefficient

Re-using the L10 §6 used-cars data ($n = 200$, $p = 8$). Bootstrap a 95% CI for the LASSO coefficient on `mileage` at $\hat{\lambda}_{\text{CV}} = 100$ (the value picked in §5.6).

Procedure ($B = 1000$):

1. Draw a bootstrap sample of 200 rows from the original data set.
2. Refit LASSO at $\lambda = 100$ on the resampled rows.
3. Record $\hat{\beta}^{*b}_{\text{mileage}}$.

Result (fabricated):

9.3 Worked Example — LASSO Coefficient 2

Quantity	Value
$\hat{\beta}_{\text{mileage}}$ (original fit)	-235
Bootstrap mean	-238
$\hat{\text{SE}}_{\text{boot}}$	31
2.5% quantile	-298
97.5% quantile	-177

95% bootstrap percentile CI: $[-298, -177]$ USD per 1,000 miles.

Why this is remarkable. LASSO has no closed-form SE formula — the ℓ_1 penalty breaks the L7/L9 derivation. The bootstrap doesn't care. The procedure above would work identically if the target were R^2, a predicted price for a specific car, or any other function of the fit.

10. Bootstrap Example 2 — Hypothesis Tests

10.1 The Idea — A P-Value From Resampling

Recall L7 §5–6: a p-value is the probability, under the null hypothesis, of seeing a statistic at least as extreme as the observed one.

Bootstrap p-value:

1. Construct a bootstrap distribution of $\hat{\theta} - \theta_0$ under H_0. (Practical recipe: subtract $\bar{\hat{\theta}}^*$ from each $\hat{\theta}^{*b}$ to centre the bootstrap distribution at zero — mimicking "the null is true".)
2. Count: what fraction of those centred bootstrap statistics are as extreme as the observed $\hat{\theta} - \theta_0$?

No closed-form null distribution required. No t-table.

10.2 Worked Example — Median Sale Price

Same used-cars data. Test whether the population median sale price equals \$15,000:

$$H_0 : m = 15000 \quad \text{vs.} \quad H_1 : m \neq 15000.$$

Observed sample median: $\tilde{m} = 14{,}320$. Observed statistic: $\tilde{m} - 15000 = -680$.

Bootstrap procedure ($B = 10000$):

1. Draw bootstrap sample. Compute its median $\tilde{m}^{*b}$.
2. Centre: $\tilde{m}^{*b} - \bar{\tilde{m}}^{*}$ (an under-H_0 replicate).

10.2 Worked Example — Median Sale Price 2

P-value:

$$\widehat{p} \;=\; \frac{1}{B}\sum_{b=1}^{B}\mathbf{1}\Big\{|\tilde{m}^{*b}-\bar{\tilde{m}}^{*}| \;\geq\; |\tilde{m}-15000|\Big\}.$$

Fabricated result: $\widehat{p} = 0.31$. Cannot reject H_0. The observed gap of \$680 is well within the range of variability the bootstrap sees in the sample median.

10.3 Generality

The recipe from §10.1 doesn't care what $\hat{\theta}$ is.

Same procedure works for testing:

- A LASSO coefficient $\hat{\beta}_j = 0$.
- $R^2 = R_0^2$ for some null value.
- Equality of predicted prices for two cars, $\hat{y}(\mathbf{x}_1) = \hat{y}(\mathbf{x}_2)$.
- Equality of medians across two groups.
- Anything else expressible as "compute a statistic from a data set".

This is what formula-free means. The L7/L9 SE formulas came from per-statistic algebra. The bootstrap reduces all of them to one recipe: resample, recompute, read off.

11. Why the Bootstrap Works — Intuition

11.1 What We Actually Want

Pull back to first principles. $\hat{\theta}$ is a statistic computed from data $Y_1, \ldots, Y_n$ drawn iid from some population distribution F.

We want to know

$$\mathrm{Var}_F(\hat{\theta}) \;=\; \mathbb{E}_F\big[(\hat{\theta} - \mathbb{E}_F[\hat{\theta}])^2\big],$$

the variance of $\hat{\theta}$ across hypothetical replicate data sets, each of size n drawn from F.

Obstacle: we have no F. We have one data set.

The bootstrap is a clever two-stage approximation that side-steps F entirely.

11.2 Layer 1 — Replace F With $\hat{F}_n$

Define the empirical distribution

$$\hat{F}_n(t) \;=\; \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{y_i \le t\}.$$

$\hat{F}_n$ puts probability mass $1/n$ on each observed y_i.

Glivenko–Cantelli (informal)
As $n \to \infty$, $\hat{F}_n$ converges to F uniformly, almost surely.

In words: the empirical distribution is the best estimate of the unknown population distribution we can build from data. For moderately large n, it is a very good estimate.

Layer 1. Replace $\mathrm{Var}_F(\hat{\theta})$ with $\mathrm{Var}_{\hat{F}_n}(\hat{\theta})$. Statistically justified.

11.3 Layer 2 — Simulate Under $\hat{F}_n$

Even with $\hat{F}_n$ in hand, computing $\mathrm{Var}_{\hat{F}_n}(\hat{\theta})$ analytically is hopeless for complicated statistics (median, LASSO coefficient, R^2).

Workaround: simulate. Draw many samples of size n from $\hat{F}_n$ and compute the sample variance of $\hat{\theta}$ across those simulations.

Drawing a sample of size n from $\hat{F}_n$ is exactly drawing n values with replacement from the observed data. That is the bootstrap resampling step.

Layer 2. Replace $\mathrm{Var}_{\hat{F}_n}(\hat{\theta})$ with the sample variance of the bootstrap replicates, $\widehat{\mathrm{SE}}^2_{\mathrm{boot}}(\hat{\theta})$. Monte Carlo justified.

11.4 The Two Layers, Side by Side

The bootstrap as two approximations

$$\underbrace{\mathrm{Var}_F(\hat{\theta})}_{\text{what we want}} \overset{\text{Layer 1}}{\approx} \underbrace{\mathrm{Var}_{\hat{F}_n}(\hat{\theta})}_{\text{conceptually defined}} \overset{\text{Layer 2}}{\approx} \underbrace{\widehat{\mathrm{SE}}^{\,2}_{\mathrm{boot}}(\hat{\theta})}_{\text{actually computed}}.$$

Layer 1 is the statistical approximation. It improves as n grows. Beyond our control once data are collected.

Layer 2 is the Monte Carlo approximation. It improves as B grows. Entirely under our control — just run more resamples.

Both approximations are cheap conceptually. Together they estimate any property of the sampling distribution of any computable statistic.

11.5 Why This Is Profound

No knowledge of F was needed.

No closed-form derivative or asymptotic expansion was needed.

The exact same three-line recipe (resample, recompute, read off) applies to:

- The sample mean (where we already had a clean SE formula — bootstrap agrees).
- The sample median (where we don't have a clean SE).
- LASSO coefficients, R^2, predicted values.
- Anything else.

The bootstrap turns statistical inference into a repetitive computational task. Computer software carries out the resampling and recomputation using the data we collected.

12. Bootstrap vs the Asymptotic Approach

12.1 The Asymptotic Recipe

Under regularity conditions, the central limit theorem (L3 §6) gives

$$\sqrt{n}\left(\hat{\theta} - \theta\right) \xrightarrow{d} N(0, V)$$

for some variance V that depends on F and on the form of $\hat{\theta}$.

Workflow:

1. Derive V by hand (delta method, sandwich formulas, score equations).
2. Substitute a sample estimate $\hat{V}$.
3. Build CIs and tests using $\hat{\mathrm{SE}} = \sqrt{\hat{V}/n}$ and a normal (or t) reference distribution.

Classical L7 inference for $\hat{\beta}_1$ is one instance of this recipe.

12.2 Where Asymptotics Fail

Three common failure modes:

- V depends on unknown nuisance functions. Example: median's $V = 1/(4f(m)^2)$ depends on the population density at the population median. We don't have $f(m)$.
- The statistic is non-smooth. Quantiles, the max, the min, ℓ_1-penalised estimators with parameters at the corners. Asymptotic normality breaks down or requires special handling.
- The data are not iid. Heteroscedastic, clustered, time-correlated — the L7 SE formula was derived under iid. Each new dependence structure requires a new derivation (sandwich, cluster-robust, Newey-West, etc.).

In all three failure modes, the bootstrap (with appropriate variants for non-iid data) still works.

12.3 The Bootstrap as a Universal Substitute

Conditions for the bootstrap to apply:

1. You can compute $\hat{\theta}$ from a data set.
2. You can draw bootstrap samples (resample with replacement).

That's it. No derivation, no formula, no regularity conditions to check by hand.

Cost: B refits. For most modern statistics on most modern data sets, that cost is trivial. Inferential generality buys you compute, and compute is cheap.

12.4 Comparison Table

Aspect	Asymptotic	Bootstrap
Needs SE formula?	yes	no
Per-statistic derivation?	yes	no
Computational cost	low	high (B refits)
Generality	low	very high
Small-sample accuracy	depends	depends on B and n
Non-iid data	re-derive	resample variant
Non-smooth statistic	breaks	works
Implementation	math	loop

Rows 1–2 are the headline: bootstrap removes per-statistic algebra. Rows 7–8 spell out where bootstrap dominates — hard cases or compute-rich environments.

12.5 Bottom Line

Both methods estimate the same population quantity — the variance of $\hat{\theta}$ under F.

Where the asymptotic route is available and tractable (sample mean, sample variance, OLS with independent observations and constant conditional variance), the two methods agree. Use the closed form; it's faster and cleaner.

Where the asymptotic route is blocked or painful, the bootstrap is the universal tool. It transforms "derive an SE formula" into "run a loop".

In modern data science workflows, the bootstrap is the default unless a closed form is obviously simpler.

13. Summary

13.1 Foundations

Three big ideas:

- Test error, not training error, is the right criterion for model selection. Training error is too optimistic because of double use of the data.
- K-fold cross-validation estimates test error generically. One procedure picks every tuning parameter we have left dangling: λ in L10 ridge/LASSO, λ in L12 LASSO logistic, d in L11 polynomial regression, subset choice in L10 best-subset.
- The bootstrap estimates the variability of *any* computable statistic by resampling the data with replacement. Two layers of approximation (empirical-for-population, simulation-for-analytic-variance) justify it. Formula-free; applies to any method in the course.

13.2 Key Formulas

- K-fold CV-MSE. $\mathrm{CV}_K = \frac{1}{K}\sum_{k=1}^{K} \mathrm{MSE}_k$.
- Selection rule. $\hat{\theta}_{\mathrm{CV}} = \arg\min_\theta \mathrm{CV}_K(\hat{f}_\theta)$.
- Bootstrap SE. $\widehat{\mathrm{SE}}_{\mathrm{boot}} = \sqrt{\frac{1}{B-1}\sum_b(\hat{\theta}^{*b} - \bar{\hat{\theta}}^{*})^2}$.
- Bootstrap percentile CI. $[\hat{\theta}^*_{(\alpha/2)}, \hat{\theta}^*_{(1-\alpha/2)}]$.
- Bootstrap p-value. fraction of centred resamples at least as extreme as the observed statistic.

Every formula is universal in $\hat{\theta}$.

13.3 Forward Pointer

Forward pointer to Lecture 14

Random forests use the bootstrap idea as a *model-fitting* device, not just an SE device.

The procedure — called bagging (bootstrap aggregation) — fits many decision trees on B bootstrap resamples of the data and averages their predictions. The averaging cancels out variance and produces a fit much more stable than any single tree.

Same resampling idea, different purpose: here the bootstrap builds the *model*, not the standard error of the model.

Closing. L13 has been the most generic lecture so far: one procedure for tuning, one procedure for uncertainty, both applicable to every method we have covered. The remaining lectures plug specific models into these procedures.